CS 331 Week 6 Notes

Subset Sum Problem

Problem Statement:

Given a set of non-negative integers $S = \{w_1, w_2, \ldots, w_n\}$ and a target value W, determine whether there exists a subset $s \subseteq S$ such that:

$$\sum_{w_i \in s} w_i \leq W.$$

as large as possible.

Example

$$w_1 = 1, \quad w_2 = 2, \quad w_3 = 3, \quad W = 5$$

Brute Force Solution: Generate all 2^n subsets of n items (where $n = 3$ in this example), calculate their weights, and choose the subset of jobs such that:

$$\sum_{i \in s} w_i \leq W.$$

w_1	w_2	w_3	Subset	Total Weight
0	0	0	$\{\}$	0
0	0	1	$\{w_3\}$	3
0	1	0	$\{w_2\}$	2
0	1	1	$\{w_2, w_3\}$	$5 \leftarrow$ **Optimal**
1	0	0	$\{w_1\}$	1
1	0	1	$\{w_1, w_3\}$	4
1	1	0	$\{w_1, w_2\}$	3
1	1	1	$\{w_1, w_2, w_3\}$	$-\infty \leftarrow$ Exceeds W

Important Note: Upon selecting a weight, w_i, the max available weight W becomes $W - w_i$.

Greedy Approach?

- **Choose weights in decreasing order:** Counterexample: $S = \{12, 11, 11\}, W = 22$. Result = 12, but the optimal solution is 22 (choosing 12 prevents the optimal solution of 11 + 11).

- **Choose weights in increasing order:** Counterexample: $S = \{1, 11, 11\}, W = 22$. Result = 12, but the optimal solution is 22 (choosing 1 prevents the optimal solution of 11 + 11).

In the weighted interval scheduling problem, picking the 'best-looking' interval right now (whether by earliest finish time, largest weight, or best ratio) can block many other intervals whose combined weight is larger. So any greedy choice can fail. That's why we use dynamic programming. It considers all possibilities and guarantees an optimal solution.

Difference Between Weighted Interval Scheduling and Subset Sum

In **weighted interval scheduling**, when considering a request n, the optimal solution $\mathrm{OPT}(n)$ either includes or excludes request n.

In **subset sum**, accepting request n does not immediately imply rejecting other requests. Instead, for the subset of requests $S \subseteq \{1, 2, \ldots, n-1\}$, there is less available weight left: $W - w_n$.

Subproblem Requirement: To find $\mathrm{OPT}(n)$, we need to know:

- The value of $\mathrm{OPT}(n-1)$.
- The best solution using a subset of the first $n-1$ items with the remaining capacity $W - w_n$.

Thus, we have a subproblem for each $i = 0, 1, \ldots, n$ and each $0 \leq w \leq W$.

Recursive Formulation

Create a recurrence.

$OPT(i, w)$: the value of the optimal sol using a subset $\{1, \ldots, i\}$ with max allowed weight W.

$$OPT(i,w) = \begin{cases} \text{exclude} \\ OPT(i-1, w) \\ \\ \text{include} \\ w_i + OPT(i-1, w - w_i) \end{cases}$$

if $w_i > W$: $OPT(i,w) = OPT(i-1, w)$

else

$$OPT(i,w) = \max(OPT(i-1, w), w_i + OPT(i-1, w - w_i))$$

Subset Sum Problem: Recurrence and Iterative Algorithm

Recurrence Relation

$$\mathrm{OPT}(i, W) = \begin{cases} \mathrm{OPT}(i-1, W), & \text{if } W < w_i, \\ \max(w_i + \mathrm{OPT}(i-1, W - w_i), \mathrm{OPT}(i-1, W)), & \text{otherwise.} \end{cases}$$

Iterative Algorithm:

SS(n, W):

as s increases by 1 bit, the runtime doubles.

- Define $M[0 \ldots n, 0 \ldots W]$.
- Initialize $M[0, w] = 0$ for each $w = 0, 1, \ldots, W$.

$s = \log_2 W$

Algorithm:

$O(n W) \equiv O(n 2^s)$

for $i = 1, 2, \ldots, n$:
  for $w = 0, 1, \ldots, W$:
    if $w < w_i$:
      $M[i, w] = M[i-1, w]$,
    else:
      $M[i, w] = \max(M[i-1, w], w_i + M[i-1, w - w_i])$.
return $M[n, W]$.

input size of a number W is the number of bits required to represent W in binary.

Time Complexity

$$O(n \cdot W) \quad \text{(pseudo-polynomial time).}$$

Note: Pseudo-polynomial algorithms can be reasonably efficient when W is small, but they become impractical as W grows larger.

The time complexity $O(n \cdot W)$ can be expressed in terms of the size of the input $s = \log W$:

$$O(n \cdot W) = O(n \cdot 2^s).$$

Here, $W = 2^s$, where s is the number of bits required to represent W.

Example:

$w_1 = w_2 = 2,\ w_3 = 3,\ W = 6$

M

set of items to choose from

	0	1	2	3	4	5	6
3	0	0	2	3	4	5	5
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0

upperbound on weight

		$M[i, w]$
$M[i-1, w-2]$	$[M i-1, w-]$	$M[i-1, w]$

Claim

The function $\mathrm{SS}(j, w)$ computes $\mathrm{OPT}(j, w)$ for each $j = 1, 2, \ldots, n$ and $w = 0, 1, \ldots, W$.

Proof of Correctness (By Induction on $j + w$)

Base Case: For $j + w = 0$, $\mathrm{OPT}(0, w) = 0$ for all $w = 1, 2, \ldots, W$, and since for $j = 0$, our algorithm will return 0, this is true.

Inductive Hypothesis: Assume that $\mathrm{SS}(j, w)$ computes $\mathrm{OPT}(j, w)$ for all $1 \leq j + w \leq k$.

Inductive Step: For $j' + w' = k + 1$, there are two cases to consider:

- **Case 1: j with $w + 1$** If we increment the weight by 1, then:
  - If $w + 1 \geq w_j$, we have two choices:
    (1) Add w_j: Since $j - 1 + w + 1 - w_j \leq k$, we already know that the optimal solution for j with allowed weight $w + 1 - w_j$ is $M[j - 1, w + 1 - w_j]$. Hence, adding w_j gives:
    $$w_j + M[j - 1, w + 1 - w_j].$$
    (2) Do not add w_j: The set remains as $M[j - 1, w + 1]$, which is already optimal since $j - 1 + w + 1 = j + w \leq k$.

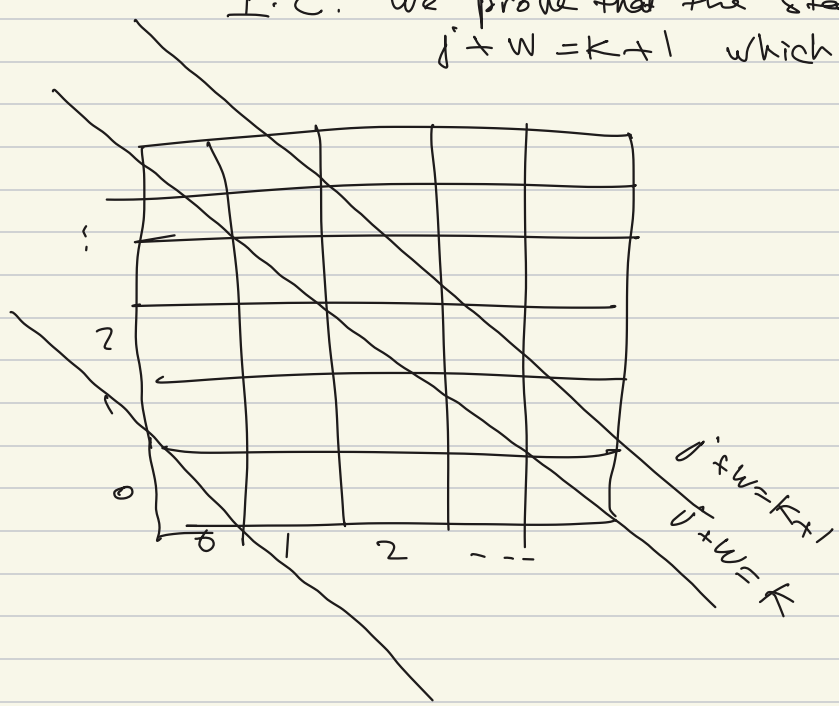
    We choose the larger of these two solutions:
    $$\max(w_j + M[j - 1, w + 1 - w_j], M[j - 1, w + 1]).$$
  - If $w + 1 < w_j$, we have only one choice: do not add w_j. In this case, the solution remains:
    $$M[j - 1, w + 1].$$

  In all cases, we guarantee the solution is optimal.
- **Case 2: $j + 1$ with w** The proof is similar to Case 1 by substituting j with $j + 1$ and substituting $w + 1$ with w.

Thus, for $j' + w' = k + 1$, $\mathrm{SS}(j', w') = \mathrm{OPT}(j', w')$, showing that for any j, w, $\mathrm{SS}(j, w)$ computes the optimal solution.

<u>Show</u> that $SS(j, w)$ computes $OPT(j, w)$
$j = 1, 2, \ldots, n$, $w = 1, 2, \ldots W$

Note: we have two var j, W
we want to show that a stat is true for all j & W
If we have a grid of j & W, this is equivalent to proving the stat is true for all cells in the grid.
I.H. we assume that the stat is true for all $j + W \leq K$, which are all the cells on or behind a diagonal.
I.C. we prove that the stat is true for $j + W = K + 1$ which is the next diagonal



Knapsack

Given a set of items with non-negative weights,

$$S = \{w_1, w_2, \ldots, w_n\},$$

where each weight has a value v_i and a maximum weight W; choose a subset $s \subseteq S$ so as to maximize

$$\sum_{i \in S} v_i$$

under the constraint

$$\sum_{i \in S} w_i \leq W.$$

Difference from Subset Sum: In the Subset Sum problem, the goal is to decide whether a subset exists whose total weight equals a given target. Here, we want to maximize the value instead of the weights.

Note: This is known as the **0/1 Knapsack** problem since you are not allowed to take fractions of items.

Subset Sum Recurrence

$$\text{If } w < w_n, \quad \text{then } \mathrm{Opt}(n, w) = \mathrm{Opt}(n-1, w).$$

Otherwise,

$$\mathrm{Opt}(n, w) = \max\{w_n + \mathrm{Opt}(n-1, w - w_n), \ \ \mathrm{Opt}(n-1, w)\}.$$

Knapsack Recurrence

$$\text{If } w < w_n, \quad \text{then } \mathrm{Opt}(n, w) = \mathrm{Opt}(n-1, w).$$

Otherwise,

$$\mathrm{Opt}(n, w) = \max\{v_n + \mathrm{Opt}(n-1, w - w_n), \ \ \mathrm{Opt}(n-1, w)\}.$$

Fractional Knapsack

If we are allowed to take fractions of items, we can solve the problem with a simple greedy algorithm.

Example:

Given a set of items with weights

$$S = \{w_1 = 1,\ w_2 = 2,\ w_3 = 3,\ w_4 = 4\},$$

values

$$V = \{v_1 = 30,\ v_2 = 40,\ v_3 = 45,\ v_4 = 100\},$$

and a maximum weight $W = 6$.

Solution:

Sort the items by the ratio

$$p_i = \frac{v_i}{w_i},$$

where a larger v_i is better and a smaller w_i is preferable.

The ratios are

$$p = \{p_1 = 30,\ p_2 = 20,\ p_3 = 15,\ p_4 = 25\}.$$

Thus, one optimal selection is to take the items corresponding to w_1 and w_4 entirely, and half of w_2:

$$S = \{w_1,\ w_4,\ \tfrac{1}{2}w_2\}.$$